An Analytical Comparison Of Tree-Based Machine Learning Techniques to Model and Predict Flight Delays

Sean P Genz,[1] Samuel R. Patterson,[2] Blake W. Seif,[3] and Nathaniel C. McIntire[4]
North Carolina State University, Raleigh, NC, 27606, USA

This study evaluates tree-based machine learning models, including Classification and Regression models using both the Gini Impurity method as well as the Entropy method, and Random Forests, for predicting flight delays. Using a Kaggle dataset with flight and weather details, models were compared based on AUC scores, ROC curves, and recall values. Results indicate Tomek-links and SMOTE balancing on a Random Forest model achieves the highest AUC (83%), demonstrating its utility for enhancing airline decision-making and operational efficiency.

Nomenclature

- AUC = Area Under Curve
- ROC = Receiver Operating Characteristic
- SMOTE = Synthetic Minority Oversampling Technique
- CART = Classification and Regression Tree

[1]Student, Department of Mechanical and Aerospace Engineering, Undergraduate Senior standing, spgenz@ncsu.edu
[2]Student, Department of Mechanical and Aerospace Engineering, Undergraduate Senior standing, srpatter@ncsu.edu
[3]Student, Department of Mechanical and Aerospace Engineering, Graduate standing, bwseif@ncsu.edu
[4]Student, Department of Mechanical and Aerospace Engineering, Undergraduate Junior standing, ncmcinti@ncsu.edu

I. Introduction

With the continuous rise and evolution of the commercial aviation industry, the need for effective and accurate variable modeling and delay prediction has become paramount for both consumers and the industry. Flight delays not only inconvenience passengers but also result in significant financial losses for airlines and associated businesses. [1],[2] Therefore, accurately predicting flight delays can enhance operational efficiency and customer satisfaction.

In recent years, machine learning techniques have been increasingly leveraged to address the complex issue of flight delay prediction. Among these techniques, tree-based models have garnered attention for their robustness and interpretability. Decision Trees, Random Forests, and Gradient Boosting Machines are known for their ability to handle non-linear relationships and interactions between variables, making them suitable for this task.

While several studies have explored the application of these models in various domains, comprehensive comparisons of their performance in the context of flight delay prediction remain limited. This research aims to fill this gap by conducting an analytical comparison of different tree-based machine learning techniques to model and predict flight delays. By evaluating models such as Gradient Boosting, Cost-Pruning, and Gradient Boosted Decision Trees, this study seeks to identify the most accurate and computationally efficient approach.

A compiled dataset containing measured weather conditions along with flight details serves as the basis for training and testing the models. This dataset includes variables such as atmospheric pressure, visibility, wind speed, precipitation, departure delay, and flight information. By incorporating these datasets, the study aims to understand the impact of both operational and environmental factors on flight schedules.

The ultimate goal of this research is to improve the prediction accuracy of flight delays, thereby enabling airlines to make proactive adjustments and minimize delays. The findings from this study are expected to provide valuable

insights into delay solutions, enhancing overall flight reliability and contributing to the efficiency of the commercial aviation industry.

II. Related Work

Several studies have explored the use of machine learning models, particularly tree-based models, for predicting flight delays influenced by weather conditions. These studies have highlighted the effectiveness of decision trees in handling the complex and nonlinear relationships found in flight and weather data.

In “Empirical Study on Airline Delay Analysis and Prediction,” researchers examined flight delays by incorporating airline and meteorological records by applying L2 regularization, gaussian naive bayes, K-nearest neighbors, decision trees and random forest models. The study concluded that random forest models were able to predict flight delays with an accuracy of 82%, emphasizing the tree-based model approach in this situation. [3]

Similarly, the research in “Airline Flight Delay Prediction Using Machine Learning Models,” evaluated the performance of seven machine learning approaches (logistic regression, K-nearest neighbor, gaussian naive bayes, decision tree, supported vector machine, random forest, and gradient boosted tree) using data from John. F. Kennedy International Airport (JFK). The evaluation of algorithms revealed that the decision tree performed the best with an accuracy of 97.77%.[4]

III. Methodology

Initially, a correlation matrix was constructed in an effort to better model and understand the correlation and relationships between the following models. A linear regression model was created to assess this and to try and find any correlation between weather parameters and delays. In this initial model, there was no strong correlation and it was deemed a linear modeling of delays was not suitable. Referencing existing literature, it has been found that decision tree models excel in this particular application, alongside derivative methods (random forest, classification

and regression trees, gradient boosting, etc.). These methods were found experimentally to be most proficient in terms of minor state accuracy. Subsequently, feature engineering was pursued to make the best use of the variables available to us, and such, such as weather data and related weather-related delays, were focused upon for validation of model efficiency. Further, it was understood as well that the removal of low correlation, unpredictable, and irrelevant features from the data set would be beneficial, further justifying the isolation of exclusively weather-related data.

For the sake of reliable and efficient testing of varied models, the following models were trained, tested, and evaluated in the same manner with a 0.2 test split. Post initial implementation and testing, research regarding optimization was pursued to effectively explore how each of these models would be realistically implemented, and ensure these optimizations were included in the final result comparison.

The following algorithms were implemented at a basic level initially and then optimized using varied optimization methods. both in combination and isolative. These models were then extensively tested for each variation to ascertain the varying effect the optimization methods had negatively, or positively, on testing metrics. Once the varying tree models were effectively fully optimized and tested, these models were extensively compared to determine the most efficient and accurate algorithms within the realm of flight-delay prediction. Additionally thought was taken to propose possible improvements that could be made to the current model landscape, for the purpose of proposed further research and improved results.

IV. Data Collection & Processing

The dataset was gathered from Kaggle, provided by the user 'BigData,' and contains a range of features such as airport details, weather data, and flight delays. The initial step in preprocessing involved trimming the dataset to focus solely on the delay_weather column, which represents whether a flight was delayed or not. Along with the delay_weather column, all relevant weather data fields, labeled as station_x, were retained to provide necessary

environmental context. This reduced the dataset size by removing irrelevant variables and ensured that only the most pertinent data was used for analysis.

Next, the dataset was sorted according to the values in the delay_weather column, which categorizes the flights into two classes: delayed and non-delayed. A key observation was the significant imbalance between these two classes, with non-delayed flights making up the majority of the data. This imbalance could bias model training, as the model would likely learn to predict the majority class more effectively, potentially ignoring the delayed flights. To address this, both classes were resampled to have an equal number of 50,000 data points each, ensuring that the dataset was balanced. This equalization helps to prevent the model from being biased toward the non-delayed class, leading to more accurate and fair predictions when training machine learning models.

V. Evaluated Tree-Based Models

1. Classification and Regression Trees (CARTs)

1a. Implementation

Classification and Regression Trees (CARTs) take entire datasets, and split them into two or more subsets based on certain features to make a decision. These are easy to visualize and interpret; however they are prone to overfitting, because of their simplicity. They are also faster to train and predict. CARTs can either measure the probability of incorrect classification (Gini) or measure the uncertainty (Entropy). In our study, a Gini model is implemented using python's sklearn package, via a classification and regression tree based algorithm. This algorithm was varied in its application via varying uncertainty measurement. Each split results in exactly two subsets, making the trees binary. Gini impurity is less complex than entropy, but is typically used more because it is more computationally efficient. CART is the most flexible tree, handling both classification and regression tasks simultaneously.

1b. Viable Optimization Methods

Tomek is an optimizing technique used to exclude any data points that are too close to the decision boundary. This increases the decision boundary which in turn decreases the probability of false positives or false negatives. This optimization was used for the Gini model, as well as Entropy. The next form of optimization used was Bayesian Optimization. This is a technique used to optimize the hyperparameters by identifying values that are new

(to find better solutions) and exploiting already known values that have given high accuracy (to fine-tune the best solutions). This allows the tree model to have less evaluations and higher accuracy. This was implemented on both Gini and Entropy models. The next optimization technique used was gradient boosting. Gradient boosting optimizes the model by taking decision trees, laying them out sequentially, and with each new tree, correcting the errors from the previous tree. This creates a much more accurate model, and this was used for both Gini and Entropy models as well. The final optimization technique used was stacking ensemble. This method is used as an alternative to gradient boosting, because instead of laying out decision trees sequentially, it takes the decision trees and lays them out parallel to each other. This allows the new "meta-model" to compare each tree simultaneously and combine the predictions of the base models using logistic regression. This improves accuracy by combining the predictions of each base model. This was also used for both models.

1c. Results

Gini

The model's performance was measured in terms of its precision, recall, f1-score, and overall accuracy score on the test dataset. This test accuracy was additionally compared to the training accuracy score to gauge overfitting. Before any optimization, the AUC of the Gini Index Cart was found to be 74%. This is a relatively poor score, however considering appropriate optimization was yet to be applied, this was an acceptable foundation for comparison.

Post introducing Tomek-links and Smote balancing, it increased to 78%, showing an expected need for data processing and refinement prior to testing to ensure clear prediction capabilities. Bayesian optimization was additionally applied to refine the hyperparameters of the CART model, in an effort to increase accuracy, which was subsequently found to be 79%. This was further increased with the application of gradient boosting, which was found to increase to 80%. Using the alternative approach of a stacking ensemble algorithm, the AUC has a similar 79%, which warrants the consideration of processing cost and speed when it comes to ascertaining the superior method.

As shown in the results below, the gradient boosting and stacking ensemble methods were found to be both very close, and of higher accuracy than other optimization methods.. Considering the processing cost of gradient boosting being relatively lower than that of ensemble methods, leads to the logical conclusion that a gradient boosted algorithm is the best option among Gini index Cart Algorithms. With this in mind, due to processing restraints this

study was incapable of ascertaining the efficacy of a Bayesian-optimized gradient-boosted Gini Cart algorithm, however the results above point towards the accuracy of such a model would likely be higher than that which we tested.

Table 1. GINI Index CART Algorithm Optimization Results

Optimization Method				Variables		AUC
Majority/Minority	0 / 1	Precision	Recall	F1-score	Support	
Unoptimized	0	0.75	0.74	0.74	10035	0.7418
	1	0.74	0.75	0.74	9965	
Smote + Tomek	0	0.78	0.78	0.78	8164	0.7815
	1	0.78	0.78	0.78	8218	
Gradient	0	0.77	0.84	0.8	8164	0.8
Boosted	1	0.83	0.75	0.79	8218	
Stacking	0	0.75	0.88	0.81	8164	0.7962
Ensemble	1	0.85	0.72	0.78	8218	
bayesian	0	0.75	0.86	0.8	8164	0.7865
optimization	1	0.84	0.71	0.77	8218	

Entropy

The performance of the entropy-based CART algorithm was evaluated using precision, recall, F1-score, and overall accuracy on the test dataset. These metrics were compared to the training dataset's accuracy to assess potential overfitting. Prior to optimization, the AUC of the Entropy CART algorithm was found to be 74%. While this is a relatively modest result, it provides a baseline for further refinements.

Following the application of Tomek-links and SMOTE balancing techniques, the AUC improved to 79%, demonstrating the importance of preprocessing in enhancing model predictions. Bayesian optimization was then employed to fine-tune the hyperparameters of the Entropy CART model, resulting in a similar AUC of 79%. When gradient boosting was applied, the accuracy was fairly similar at 78%. Meanwhile, an alternative stacking ensemble method achieved a comparable AUC of 79%, indicating that both approaches yielded similar results but with

differing computational costs and complexities. These results show that every optimization method used here yields similar results. This is likely due to overfitting when implementing Tomek-links and Smote balancing. Being that the results are similar, the best possible optimization technique to utilize would be the Bayesian Optimization, due to the fact that it utilizes less computational power, but yielding similar results.

As per the results below, gradient boosting and stacking ensemble methods provided superior accuracy compared to other optimization techniques. However, given the lower computational cost associated with gradient boosting, it emerges as the more practical choice for optimizing entropy-based CART algorithms. That said, due to resource limitations, this study was unable to evaluate the potential performance of a Bayesian-optimized gradient-boosted entropy CART model. Nevertheless, based on the observed results, it is reasonable to infer that such a model would likely achieve higher accuracy than those tested in this study.

Table 2. Entropy Based CART Algorithm Optimization Results

Optimization Method				Variables		AUC
Majority/Minority	0 / 1	Precision	Recall	F1-score	Support	
Unoptimized	0	0.75	0.73	0.74	10035	0.7432
	1	0.73	0.76	0.75	9965	
Smote + Tomek	0	0.79	0.79	0.79	8216	0.7873
	1	0.79	0.79	0.79	8218	
Bayesian	0	0.74	0.86	0.8	8164	0.7873
Optimization	1	0.83	0.71	0.76	8218	
Gradient	0	0.75	0.85	0.8	8164	0.7821
Boosted	1	0.83	0.72	0.77	8218	
Stacking	0	0.79	0.79	0.79	8216	0.7873
Ensemble	1	0.79	0.79	0.79	8218	

2. Random Forest

2a. Implementation

Random Forests, a specific ensemble learning method, is built upon the simplicity of individual Classification and Regression Trees (CARTs) to overcome their limitations, such as overfitting. By constructing multiple CARTs on random subsets of the data and aggregating their predictions, Random Forests provide higher robustness and generalization capabilities. While individual CARTs are faster to train and predict, Random Forests sacrifice some computational efficiency for significantly improved accuracy and stability. Each tree in the forest splits datasets into two subsets per decision, maintaining the binary nature of CARTs.

In this study, Random Forest models were implemented using Python's sklearn package, with the possibility of leveraging both Gini impurity and Entropy as uncertainty measures to evaluate their respective impacts on model performance. Gini impurity, due to its computational simplicity, is generally favored, and ultimately focused. With this in mind however, Entropy offers additional sensitivity to class imbalances by prioritizing reductions in uncertainty and warrants further research and testing in the future.. Additionally, Random Forests inherently handle classification and regression tasks effectively by averaging predictions for regression or applying majority voting for classification.

2b. Viable Optimization Methods

Random forests are slightly more limited in their ability to be optimized, leaving one with less mobility in terms of optimization options, but increased accuracy due to its inherent benefits as an ensemble method. Firstly, Smote and Tomek class balancing were implemented to prune any unpredictable or indistinguishable points from the training set. Further, bayesian optimization was performed to optimize internal hyperparameters to increase the potential accuracy of the model via iterating the algorithm over varying hyperparameter values in order to find the ideal values, additionally helping to combat any potential overfitting as a result of poor hyperparameters. Finally, gradient boosting was performed in an effort to increase the accuracy of the model and further account for class imbalance due to the nature of the dataset.

2c. Results

The model's performance was measured in terms of its precision, recall, f1-score, and overall accuracy score on the test dataset. This test accuracy was additionally compared to the training accuracy score to gauge overfitting.

Before any optimization, the AUC of the Random Forest was found to be 78%. This is an appropriate score with no optimization. It is approximately 4% more accurate than the CART model, pre-optimization.

After implementing Tomek-links and Smote balancing, the accuracy spikes to 83%. This demonstrates how much more accurate Random Forest Decision Trees are compared to CARTs. This is 4% more accurate than CARTs with Tomek and Smote. Bayesian optimization was then utilized to optimize the hyperparameters, surprisingly yielding a much smaller AUC of 78%. This is actually less accurate than the CART when optimized with Bayesian Optimization. When gradient boosting was applied, the accuracy was fairly similar at 77%. These results show that Tomek-links and Smote balancing is the superior method of optimization in this specific case. This could be due to overfitting from gradient boosting. Gradient boosting can also increase complexity in the model. If the data is well-balanced, which it is, the additional complexity may not provide any additional benefits, and could actually detract from the performance.

Table 3. Random Forest Algorithm Optimization Results

Optimization Method				Variables		AUC Score
Majority/Minority	0 / 1	Precision	Recall	F1-score	Support	
Unoptimized	0	0.76	0.8	0.78	8164	0.7754
	1	0.79	0.75	0.77	9965	
Smote + Tomek	0	0.82	0.84	0.83	8164	0.8268
	1	0.84	0.81	0.82	8218	
Bayesian	0	0.74	0.86	0.8	8164	0.7833
Optimization	1	0.84	0.71	0.77	8218	
Gradient	0	0.72	0.86	0.78	8164	0.77
Boosted	1	0.83	0.68	0.74	8218	
Gradient	0	0.72	0.86	0.78	8164	0.77
Boosted	1	0.83	0.68	0.74	8218	

VI. Results and Analysis

As per the comparison below, random forest methods present reasonably higher accuracy among unoptimized models, however seem to become susceptible to overfitting as a result of the large processed dataset. Further analysis and testing in regards to combating overfitting is suggested, however more promisingly, would be the integration of hyperparameter optimization methods in tandem with gradient boosting methods as seen with other models. This could present promisingly higher results as compared to initial findings.

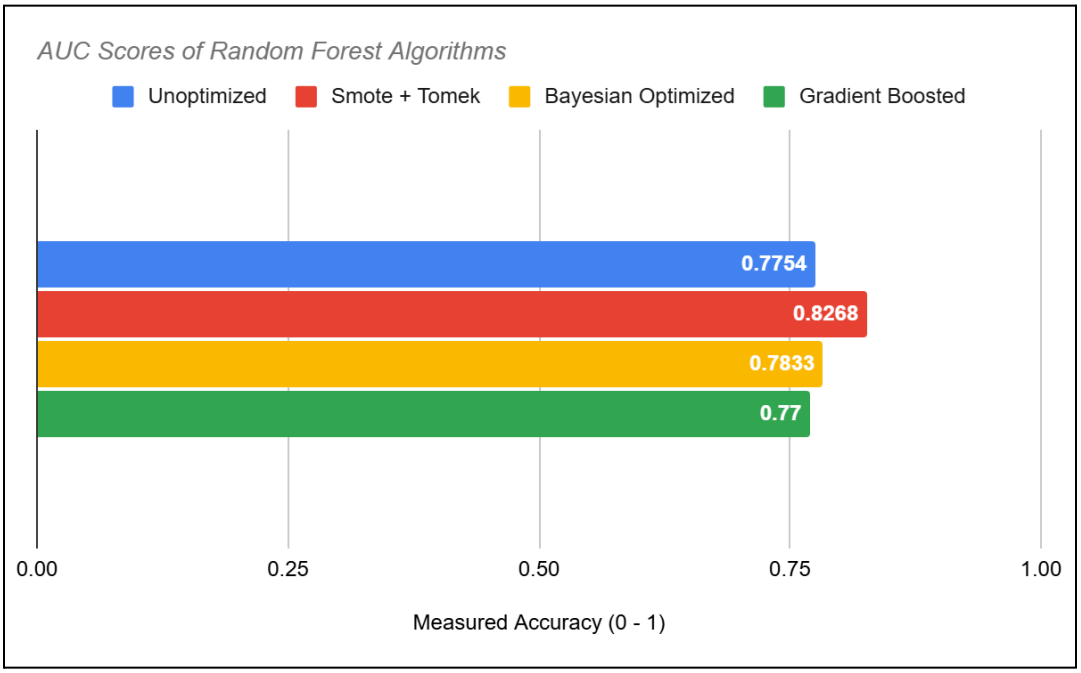

Figure 1. Accuracy Comparison of Random Forest Optimization Methods

Further, Classification and Regression tree models governed by the respective Gini Index and Entropy equations show promising results between stacking ensemble and gradient-boosted methods. Processing cost taken into consideration, however, leads to the favoring of the gradient-boosted model instead as a more cost-effective and similar performance method. Similarly to previous results regarding the random forest ensemble method, due to computational restrictions the integration of hyperparameter optimization and gradient-boosting was unable to be performed and tested accurately. With this in mind, however, the respective results of both optimizations point towards the value of combining these optimizations to improve results and accuracy further. Overall these results present lower overall performance than highly optimized models. However comparison of these models still hold valuable information, presenting by comparative numerical performance, certain methods of modeling prove more effective in this for classification tasks such as flight delay prediction.

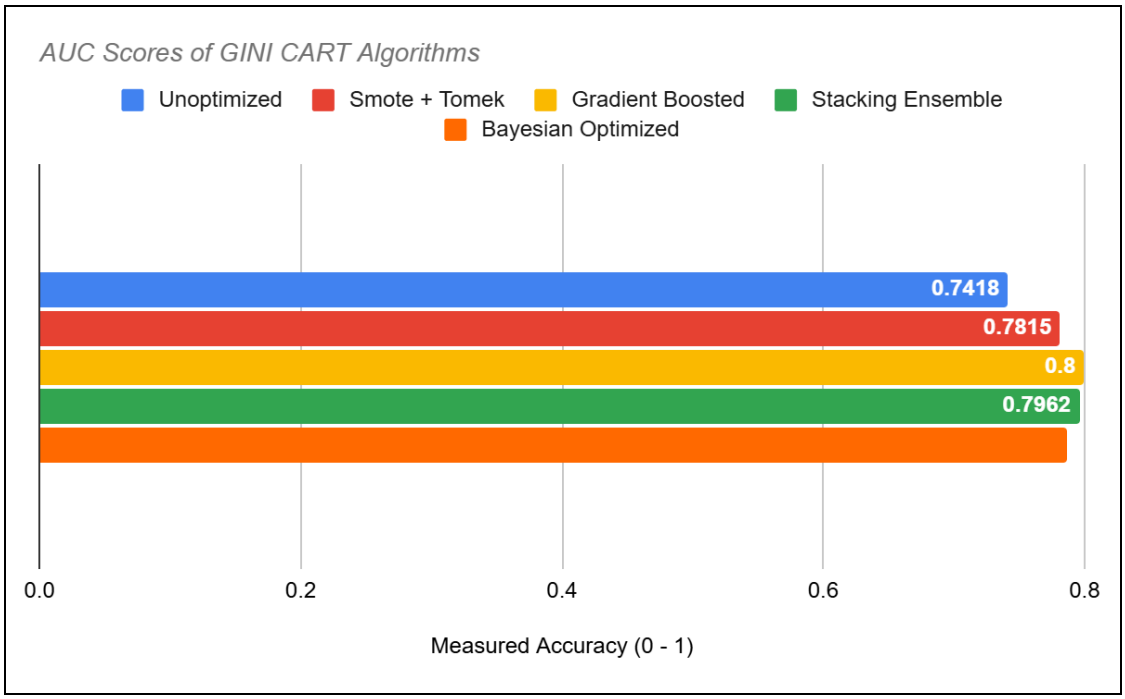

Figure 2. Accuracy Comparison of GINI CART Optimization Methods

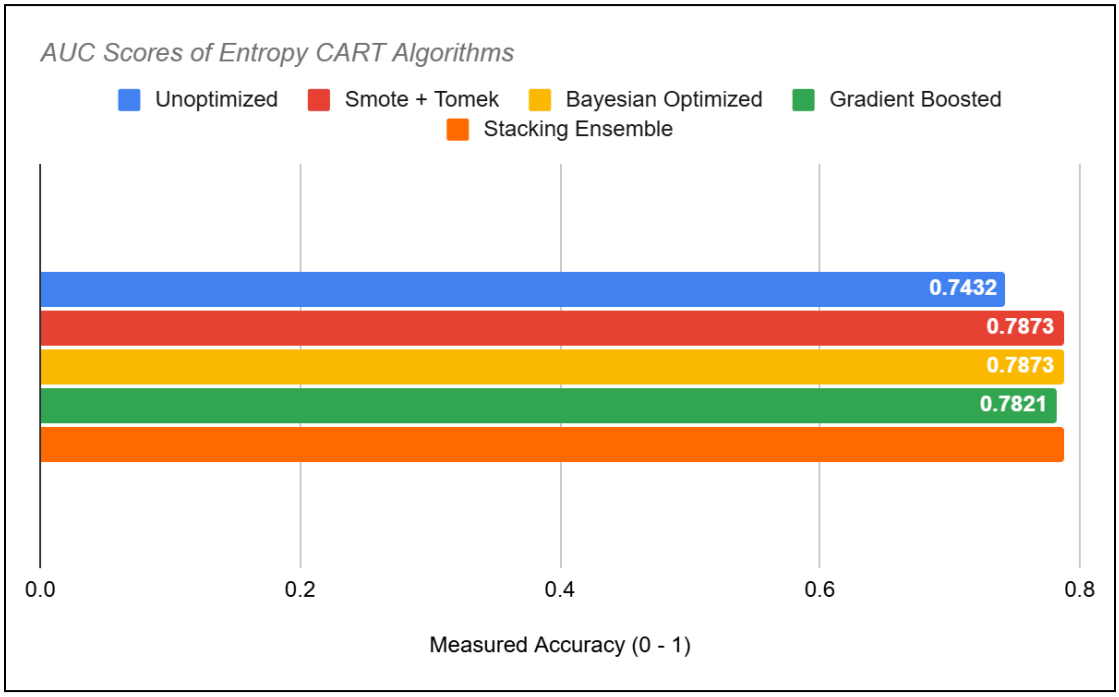

Figure 3. Accuracy Comparison of Entropy CART Optimization Methods

VII. Conclusion

This study demonstrates the capability of tree-based models to predict flight delays with high accuracy by utilizing weather and flight data. Different optimization techniques, such as SMOTE balancing, Tomek-links, Bayesian optimization, gradient boosting, and stacking ensemble, helped improve the model performances, especially when addressing data imbalances and adjusting hyperparameters.

The Gini-based CART algorithm showed initial results by achieving an AUC of 74%, which is a relatively low score, but provided an adequate basis for comparison. Once Tomek-links and SMOTE balancing was introduced the AUC improved to 78%, showing the importance of preprocessing in correcting class imbalances and enhancing prediction accuracy. Bayesian optimization further improved the AUC to 79%. However, with the addition of Gradient Boosting the AUC increased to 80% and using a stacking ensemble algorithm had a similar AUC of 79%. Since the stacking ensemble algorithm has higher computational cost this makes gradient boosting the more practical choice.

The Entropy-based CART started off with a base AUC of 74%, similar to the Gini-based CART algorithm. The AUC increased to 79% after Tomek-links and SMOTE balancing was introduced, emphasizing the significance of preprocessing in model predictions. Bayesian optimization achieved a comparable AUC of 79%. Gradient Boosting attained a lower AUC of 78%, while the stacking ensemble method was able to reach a AUC of 79%. All the optimization methods resulted in a similar AUC, but the lower computation cost of gradient boosting is the most feasible option for entropy-based CART algorithms.

The Random Forest algorithm had an AUC of 78% prior to any optimization, which is 4% higher than the pre-optimized CART models. The AUC increased to 83% after the implementation of Tomek-links and SMOTE balancing. However, the Bayesian optimization reduced the AUC to 78% and gradient boosting also resulted in the AUC to decrease to 77%, which suggests potential overfitting and unnecessary complexity in these algorithms.

Overall, tree-based machine learning models, particularly Gradient Boosting, effectively predict flight delays, achieving an AUC of 83%. Optimizations like SMOTE and Tomek balancing improve model performance,

highlighting the importance of preprocessing. Future research could explore additional data sources, such as operational metrics, to further enhance accuracy and scalability.

References

1. AirHelp, “The Economic Impact of Flight Disruptions on Airlines and Passengers,” AirHelp, 2023, pp. 1–15.
2. Airlines for America, “U.S. Passenger Carrier Delay Costs,” Airlines for America, 2023, pp. 1–10
3. Patgiri, R., Hussain, S., and Nongmeikapam, A., “Empirical Study on Airline Delay Analysis and Prediction,” *EAI Endorsed Transactions on Scalable Information Systems*, Vol. XX, No. XX, Feb. 2020, pp. 1–8. doi:10.4108/XX.X.X.XX.
4. Gupta, V., and Dutta, R., “Airline Flight Delay Prediction Using Machine Learning Models,” *Proceedings of the ACM Southeast Conference*, ACM, New York, 2022, pp. 1–7. doi:10.1145/3497701.3497725.